Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

ЗВІТ

з лабораторної роботи №1
з дисципліни «Технології та методика програмування вбудованих систем»

«Дослідження моделі пристрою керування кроковим двигуном»

Варіант №4

Виконав:
студент 1 курсу, групи ІМ-51мн
Ковальов Олександр

Перевірив:
ст. викладач, к.т.н Орленко Сергій Петрович

Дата здачі: 02.03.2026

КИЇВ – 2026

Тема. Моделювання пристрою керування кроковим двигуном з використанням мікроконтролера ATMega128.

Мета. Користуючись пакетом SimulIDE дослідити роботу пристрою керування кроковим двигуном.

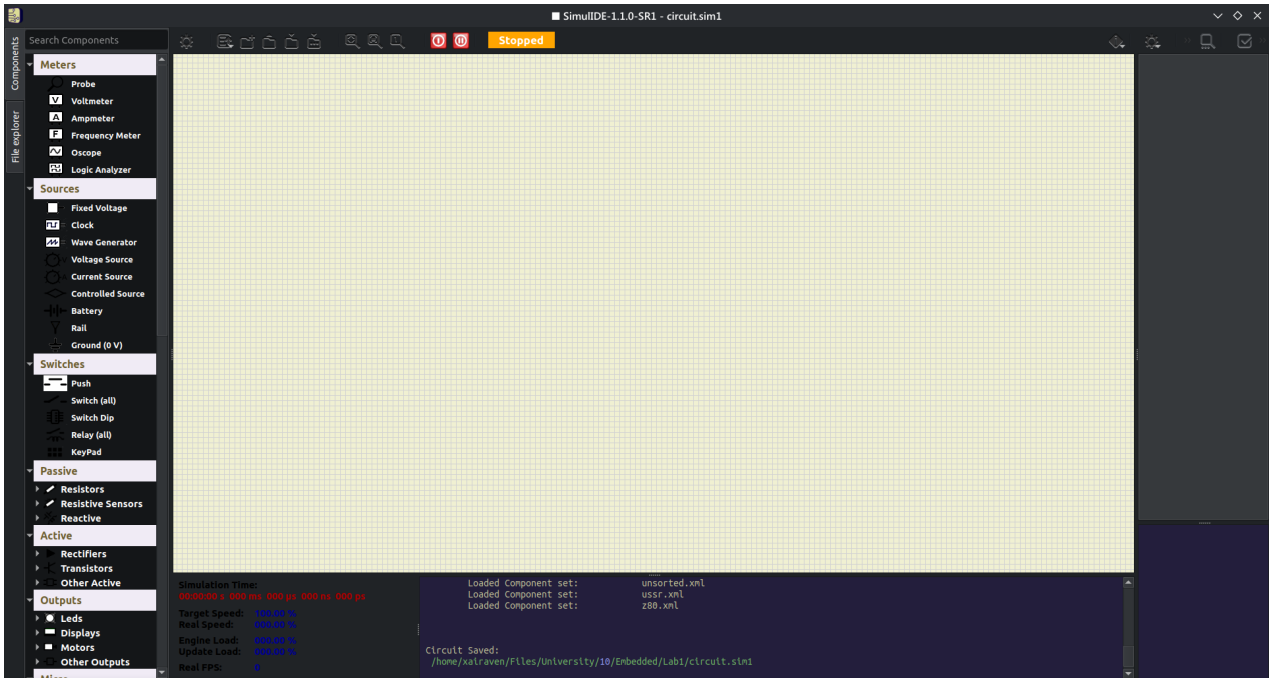
Завдання. Необхідно провести моделювання двох типів крокових двигунів: уніполярного та біполярного. Для уніполярного крокового двигуна потрібно побудувати схему з використанням драйвера ULN2003A та дослідити його функціонування у кроковому режимі роботи, а також у напівкроковому режимі. Для біполярного крокового двигуна необхідно зібрати схему на базі драйвера L293D та проаналізувати його роботу в однофазному режимі з цілим кроком, а також у напівкроковому режимі. У кожному з випадків вимагається зафіксувати керуючі послідовності імпульсів, використовуючи віртуальний осцилограф.

Звіт по роботі повинен містити:

1. схему моделі;
2. схему алгоритму роботи моделі;
3. робочу програму;
4. формули за потреби.

Хід роботи.

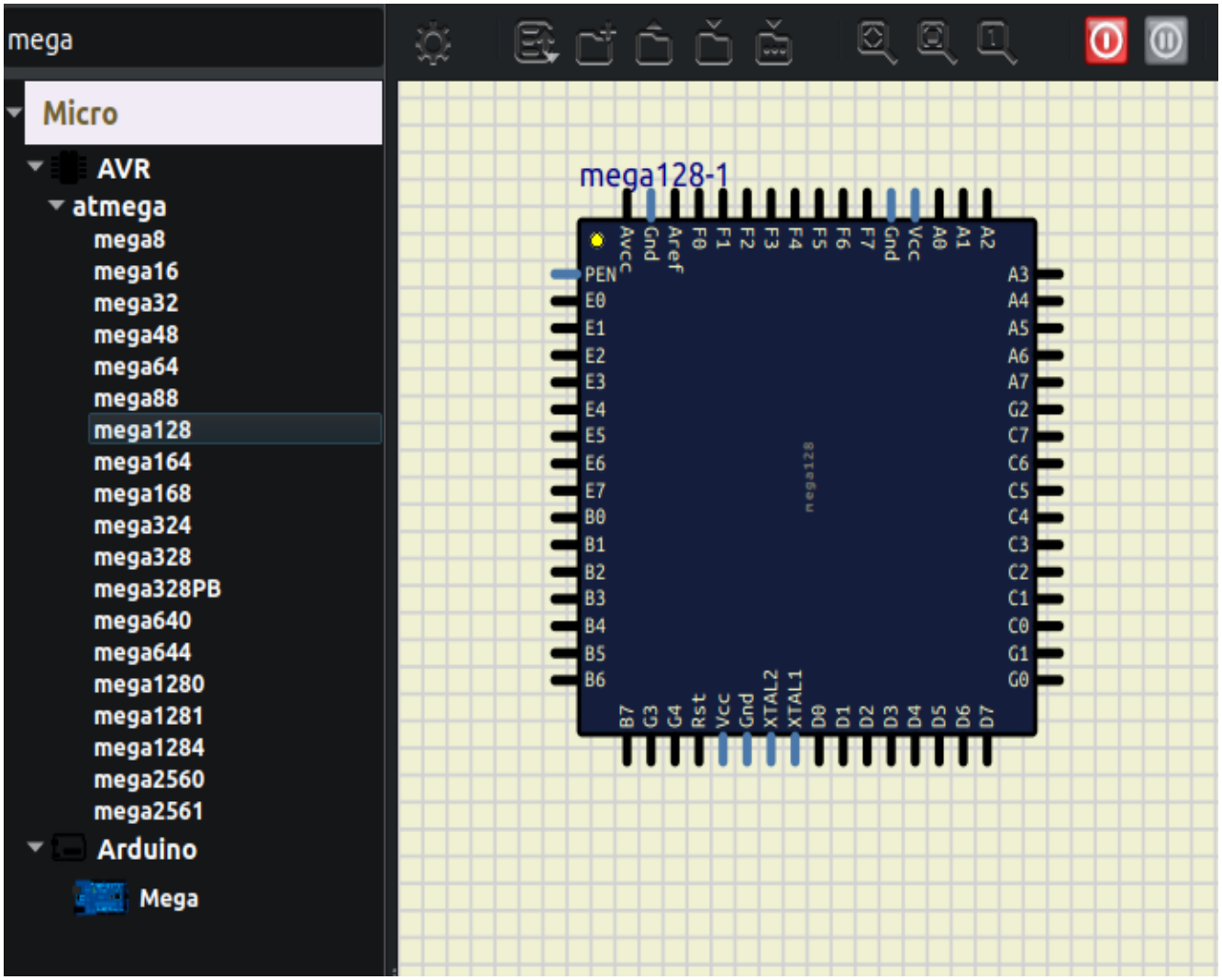
Для початку, був встановлений пакет SimulIDE, який був обраний замість Proteus VSM з декількох причин: по-перше, він безкоштовний, а по-друге, є нативна версія для системи Linux Fedora 43. Це є досить значними перевагами для автора лабораторної роботи, тому був зроблений саме такий вибір.



Щоб симулятор міг компілювати код мовою C, був встановлений компілятор `avr-gcc`:

```
xairaven@laptop ~
$ sudo dnf install -y avr-gcc avr-libc
Repositories loaded.
Package                                  Arch       Version                                  Repository                Size
Installing:
 avr-gcc                                 x86_64     1:15.2.0-1.fc43                          updates               410.1 MiB
 avr-libc                                noarch     2.2.1-3.fc43                             fedora                107.3 MiB
Installing dependencies:
 avr-binutils                            x86_64     1:2.45-1.fc43                            fedora                 10.7 MiB

Transaction Summary:
 Installing:         3 packages

Total size of inbound packages is 47 MiB. Need to download 47 MiB.
After this operation, 528 MiB extra will be used (install 528 MiB, remove 0 B).
[1/3] avr-libc-0:2.2.1-3.fc43.noarch                                          100% |   1.3 MiB/s |   2.6 MiB |  00m02s
[2/3] avr-binutils-1:2.45-1.fc43.x86_64                                       100% | 952.4 KiB/s |   2.1 MiB |  00m02s
[3/3] avr-gcc-1:15.2.0-1.fc43.x86_64                                          100% |   6.6 MiB/s |  42.6 MiB |  00m06s
--------------------------------------------------------------------------------------------------------------------------
[3/3] Total                                                                   100% |   7.1 MiB/s |  47.3 MiB |  00m07s
Running transaction
[1/5] Verify package files                                                    100% |  19.0   B/s |   3.0   B |  00m00s
[2/5] Prepare transaction                                                     100% |   4.0   B/s |   3.0   B |  00m01s
[3/5] Installing avr-binutils-1:2.45-1.fc43.x86_64                            100% | 224.5 MiB/s |  10.8 MiB |  00m00s
[4/5] Installing avr-gcc-1:15.2.0-1.fc43.x86_64                               100% | 619.8 MiB/s | 410.3 MiB |  00m01s
[5/5] Installing avr-libc-0:2.2.1-3.fc43.noarch                               100% | 114.8 MiB/s | 107.9 MiB |  00m01s
Complete!
xairaven@laptop ~
$
```

Почнемо моделювання уніполярного крокового двигуна. Був доданий мікроконтролер ATMega128:

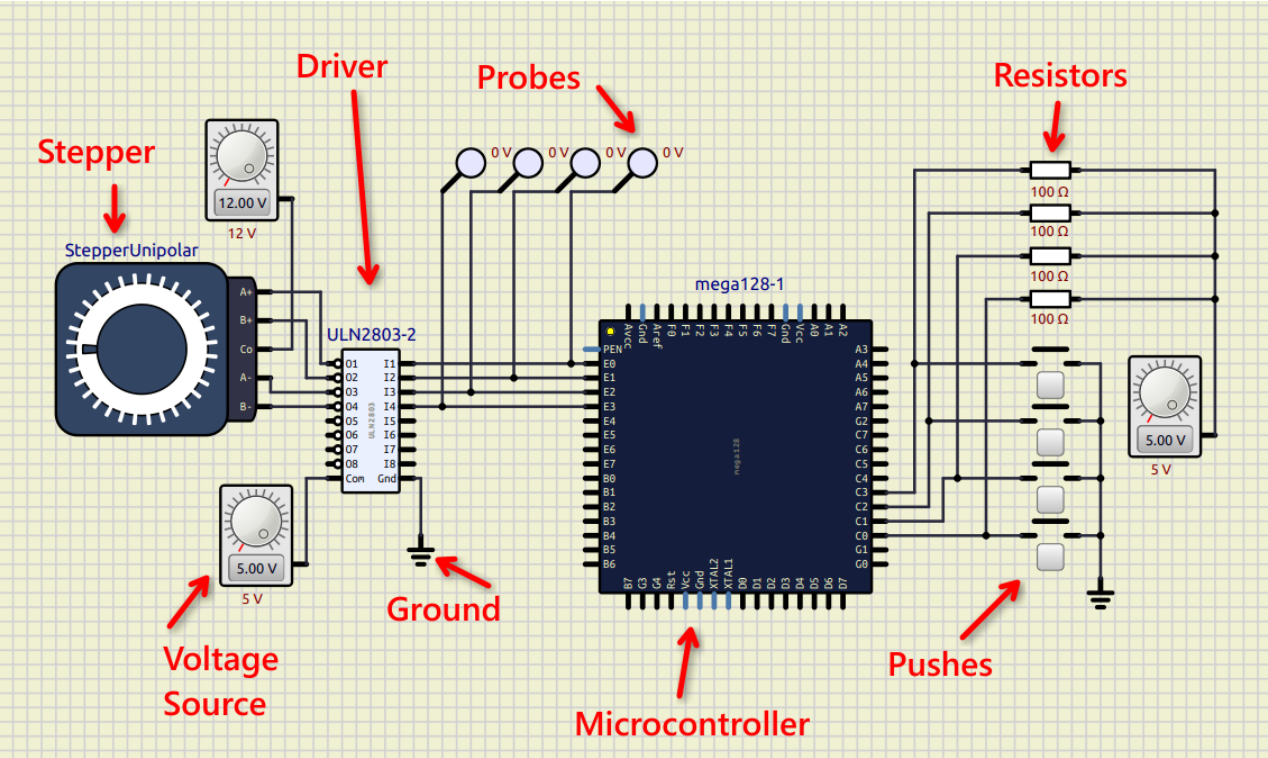


Далі треба використати драйвер ULN2003A, але в наявності бібліотеки SimulIDE є лише ULN2803 та ULN2001. Останню краще не використовувати у цій схемі. Справа в тому, що технічно вона не має вбудованих базових резисторів, які необхідні для прямої роботи зі стандартними п’ятивольтовими логічними сигналами, що надходять від мікроконтролера, тоді як модель ULN2803 є повноцінною і повністю сумісною заміною для завдання.

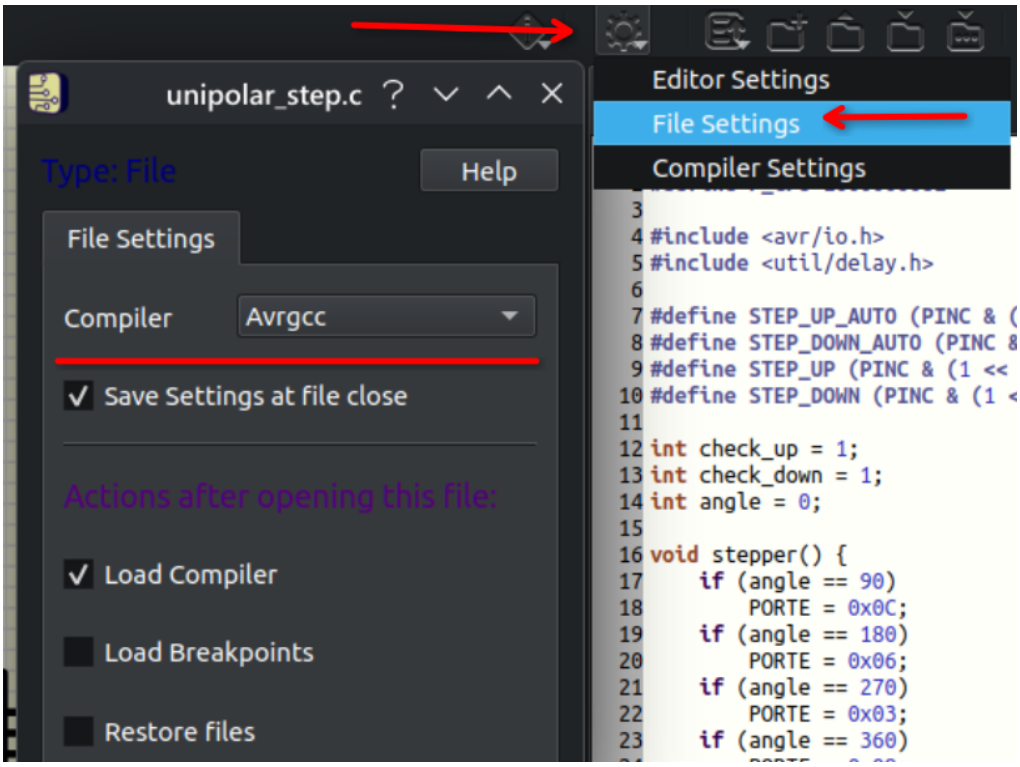
Також, для схеми потрібні інші елементи: сам уніполярний двигун, чотири кнопки, чотири підтягуючі резистори, осцилограф, джерело напруги, заземлення.

Через специфіку середовища, треба навести назви компонентів.

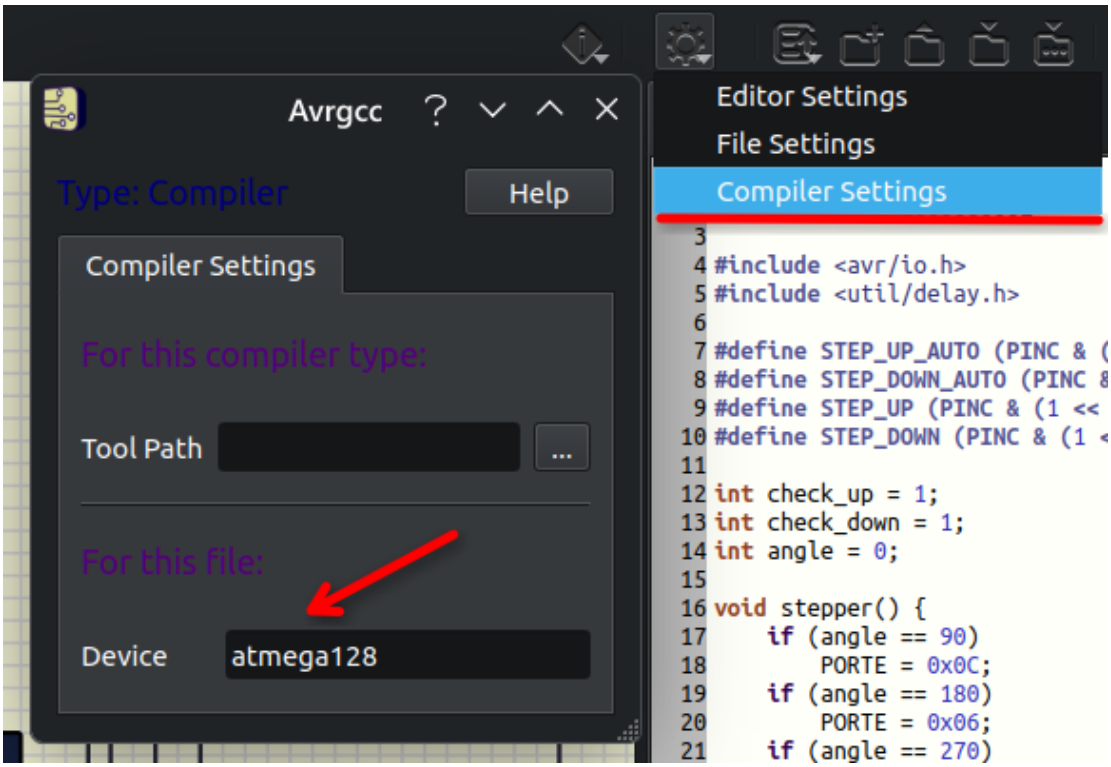
- Мікроконтролер, як і зазначено вище, підписаний «`mega128`».
- Драйвер – «`ULN2803`».
- Уніполярного двигуна (поділу) в середовищі немає, тому був використаний компонент «Stepper». Він за замовчуванням уніполярний, щоб зробити з нього біполярний, треба натиснути ПКМ, перейти в меню Parameters, і натиснути галочку «Bipolar».
- Кнопки – «Push», знаходяться в категорії «Switches».
- Заземлення – «Ground».
- Резистор – відповідно, «Resistor».
- Джерело напруги – «Voltage Source». За замовчуванням, максимальне значення теж 5 В. Це можна змінити в налаштуваннях, як і з двигуном.
- Осцилограф називається «Oscope», також використовується система щупів – «Probe».



Після цього, треба створити файл прошивки. Для цього треба налаштувати компілятор. Для початку, треба натиснути на значок шестерні, потім обрати пункт «File Settings», і обрати потрібний компілятор – `avr-gcc`.



Після цього, треба обрати пункт «Compiler Settings». В поле «Tool Path» нічого вводити не треба, так як потрібні пакети вже встановлені, компілятор можна викликати без вказання шляху – змінна оточення `PATH` налаштована. А от в поле «Device» треба написати назву мікроконтролеру – відповідно, `atmega128`.



Все готово до компіляції коду. Спочатку, буде протестований код для крокового режиму.

unipolar_step.c

```c
// Define CPU frequency as 16 MHz for the delay library
#define F_CPU 16000000UL

#include <avr/io.h>
#include <util/delay.h>

#define STEP_UP_AUTO (PINC & (1 << PINC0))
#define STEP_DOWN_AUTO (PINC & (1 << PINC1))
#define STEP_UP (PINC & (1 << PINC2))
#define STEP_DOWN (PINC & (1 << PINC3))

int check_up = 1;
int check_down = 1;
int angle = 0;

void stepper() {
    if (angle == 90)
        PORTE = 0x0C;
    if (angle == 180)
        PORTE = 0x06;
    if (angle == 270)
        PORTE = 0x03;
    if (angle == 360)
        PORTE = 0x09;
    if (angle == 0)
        PORTE = 0x09;
    if (angle == -90)
        PORTE = 0x03;
    if (angle == -180)
        PORTE = 0x06;
    if (angle == -270)
        PORTE = 0x0C;
    if (angle == -360)
        PORTE = 0x09;
}

void init_ports() {
    DDRC = 0x00;
    DDRE = 0xFF;
}

int main(void) {
    init_ports();
    while (1) {
```

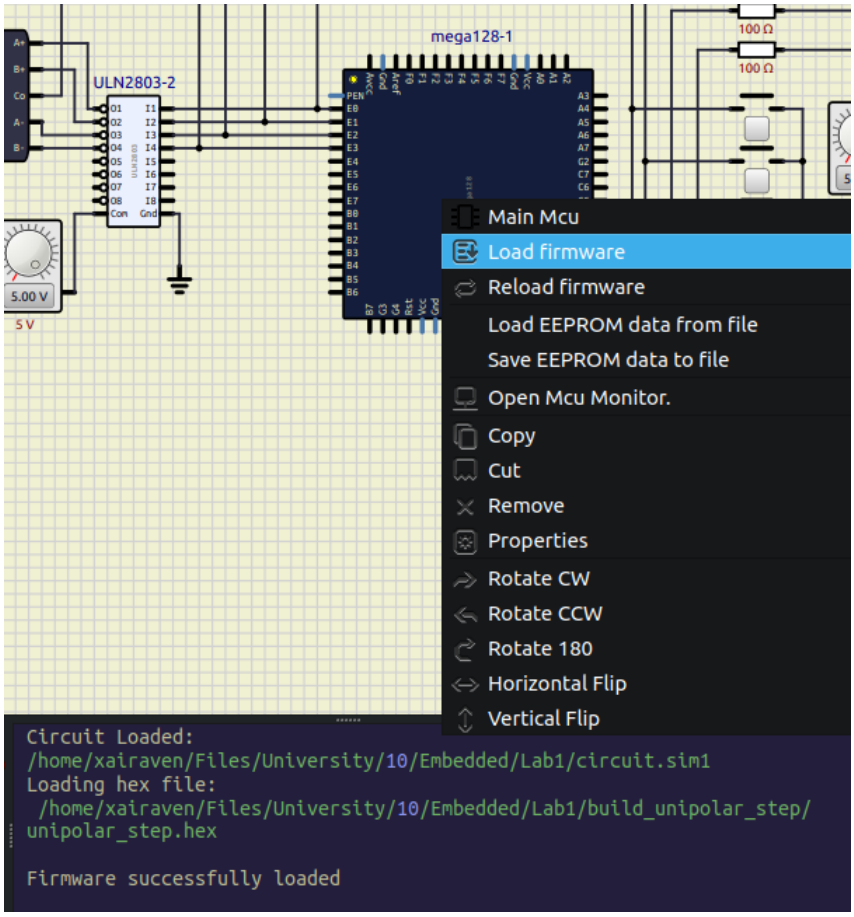
```
-------------------------------------------------------
Executing:
"avr-gcc" -mmcu=atmega128 -Wall -g -Os -o "/home/xairaven/Files/University/
10/Embedded/Lab1/build_unipolar_step/"unipolar_step.elf "/home/xairaven/
Files/University/10/Embedded/Lab1/unipolar_step.c"

Executing:
"avr-objcopy" -j .text -j .data -O ihex "/home/xairaven/Files/University/10/
Embedded/Lab1/build_unipolar_step/"unipolar_step.elf "/home/xairaven/Files/
University/10/Embedded/Lab1/build_unipolar_step/"unipolar_step.hex
```

В каталозі поряд з кодовим файлом буде створена директорія з артефактами. А саме, файл формату `.elf` та файл зі шістнадцятковими інструкціями.

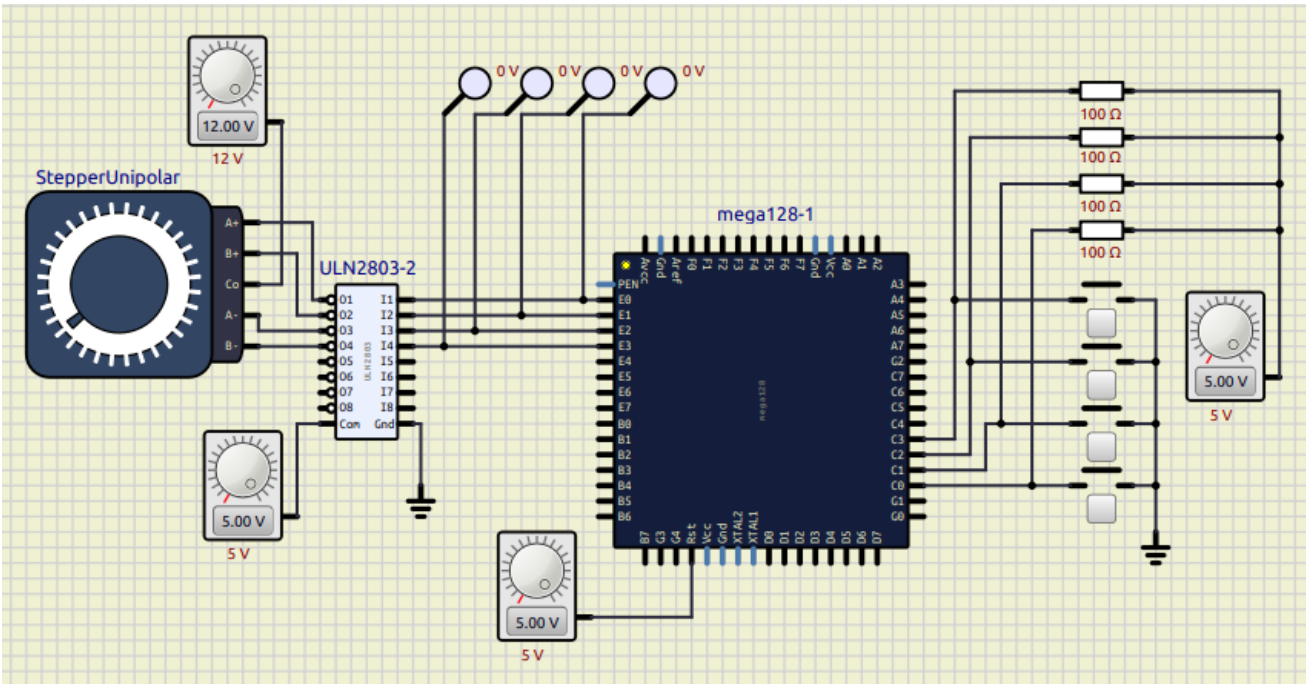
Files > University > 10 > Embedded > Lab1 > **build_unipolar_step**

Name
unipolar_step.elf
unipolar_step.hex

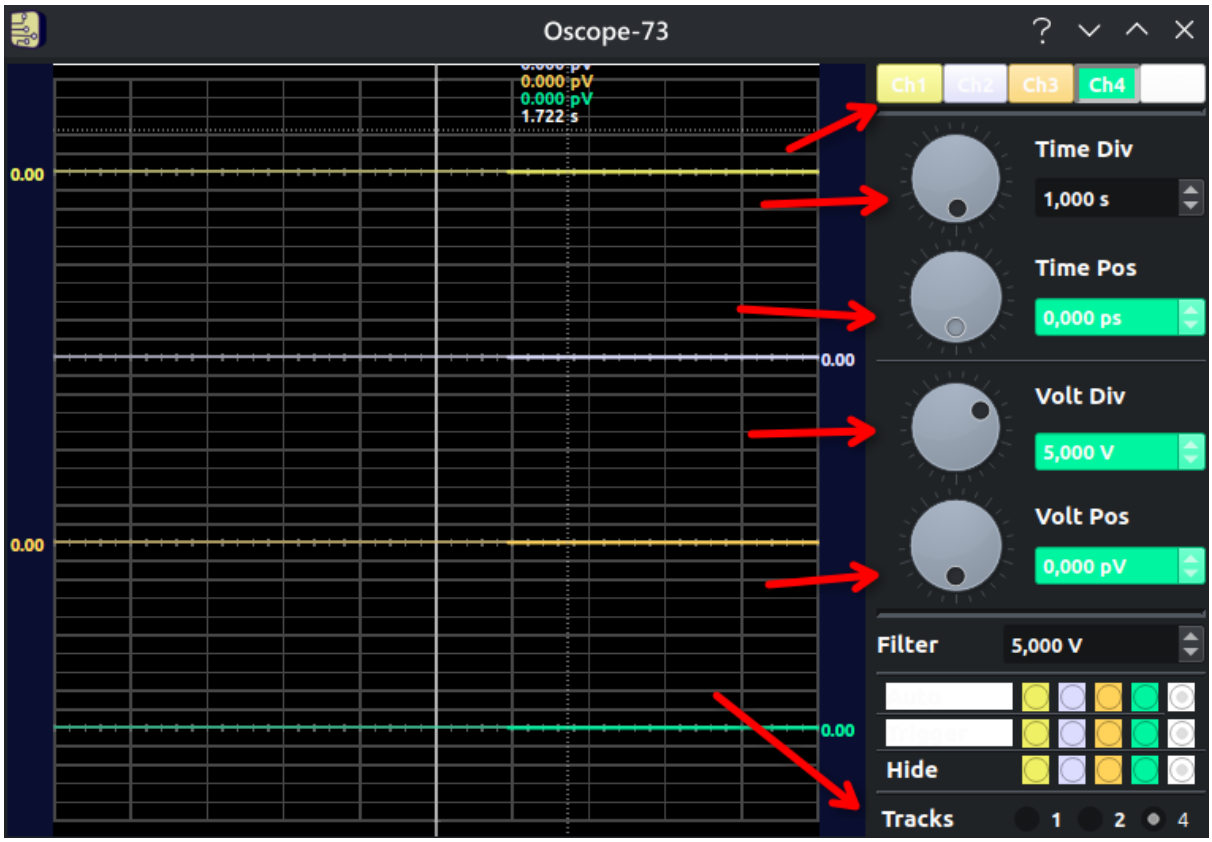
Коли є готова прошивка, можна завантажити її на мікроконтролер. Для цього треба натиснути ПКМ по ньому, обрати пункт «Load Firmware», і потім завантажити щойно скомпільований шістнадцятковий файл. Згодом, в логах можна буде побачити результат – чи пройшов процес успішно.



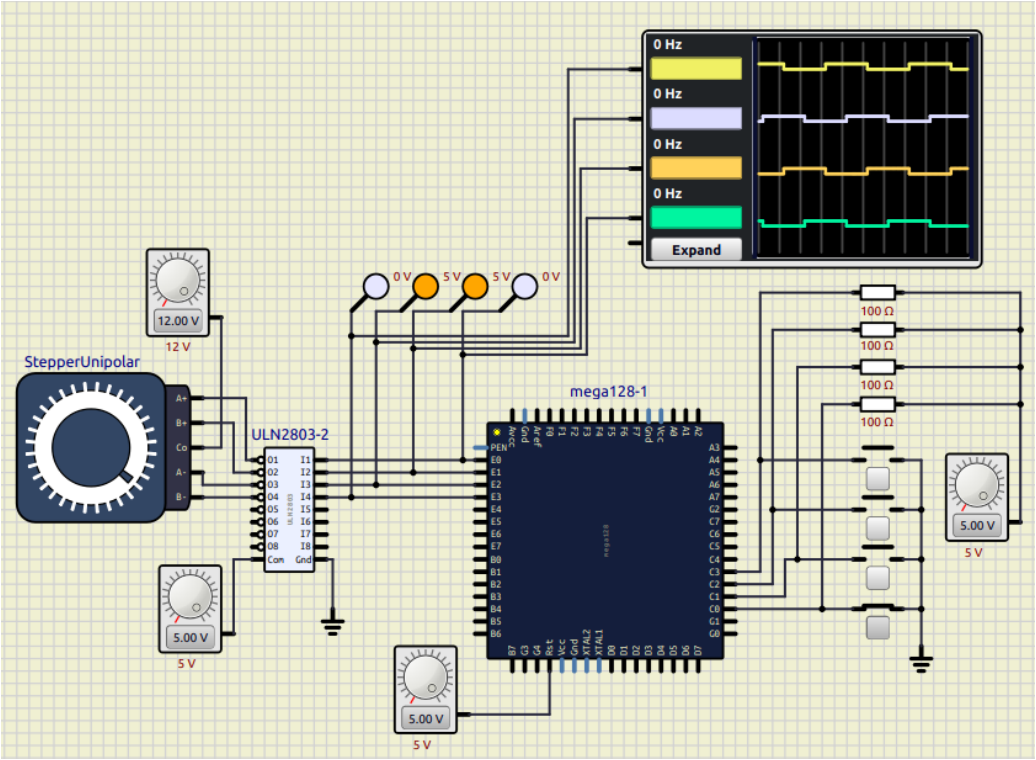
Фінальна схема (з уніполярним двигуном):



Відповідно, для гарного відображення даних на осцилографі, бажано його налаштувати. Треба виставити кількість дорожок (Tracks) – 4, і потім для кожної дорожки виставити параметри «Time Div» – 1 секунду (бо так прописано в коді), «Time Pos» – 0 (бо позиції вже налаштовані), «Volt Div» – 5 (бо така встановлена напруга), «Volt Pos» – 0 (бо знову ж, позиції вже налаштовані).

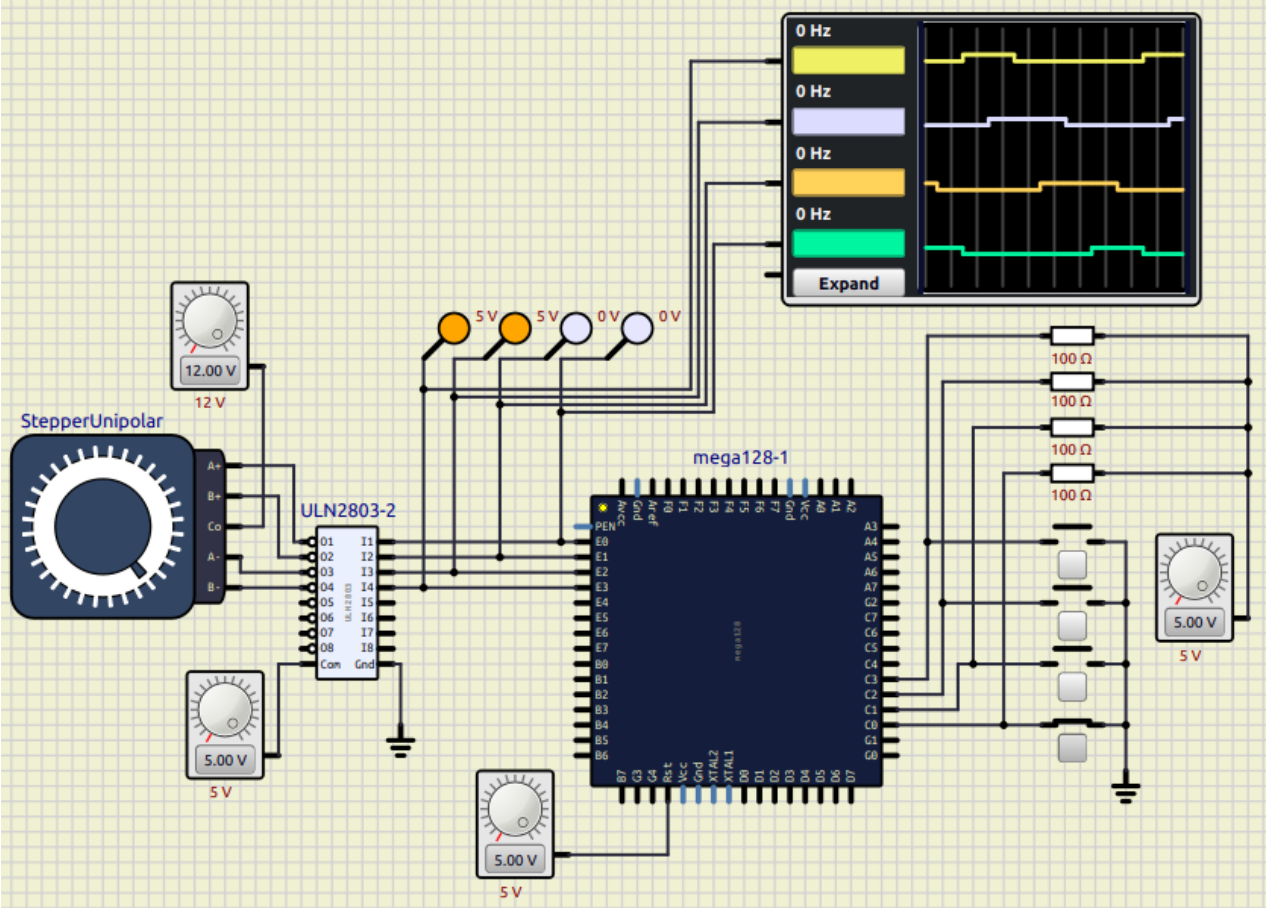


Отримуємо дані щодо крокового режиму одночасно з щупів та осцилографу:



Код – в лістингу, підписаний «`unipolar_step.c`».

Відповідно, в лістингу, підписаному «`unipolar_half_step.c`» – код для півкрокового режиму. Результат – на скріншоті:



Аналізуючи програмний код, варто відзначити кілька важливих архітектурних рішень. По-перше, початкова ініціалізація глобальних змінних (`check_up`, `check_down`, `angle`) була перенесена всередину функції `main()`. Це зроблено для обходу відомої помилки симулятора SimulIDE (`ELPM with no RAMPZ`), яка виникає при копіюванні ініціалізованих даних з Flash-пам'яті в RAM для мікроконтролера ATMega128.

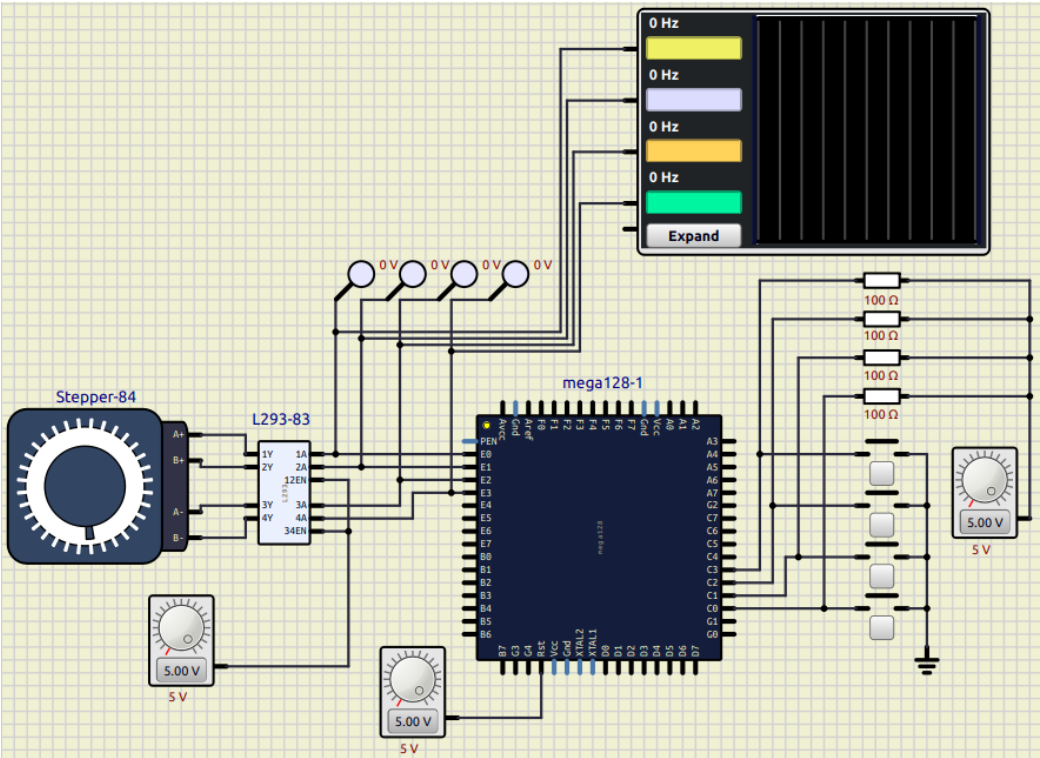
Порт C (`DDRC = 0x00`) налаштовано як вхід для зчитування стану кнопок керування, а порт E (`DDRE = 0xFF`) – як вихід для передачі керуючих сигналів на драйвер двигуна.

Для крокового режиму (full-step) уніполярного двигуна використовується послідовність збудження двох фаз одночасно. У коді це реалізовано через подачу шістнадцяткових значень на порт E: `0x0C` (двійкове 00001100), `0x06` (00000110), `0x03` (00000011) та `0x09` (00001001). Завдяки затримці `_delay_ms(1000)` між перемиканнями, ми бачимо на осцилографі чіткі прямокутні імпульси тривалістю 1 секунда.

У напівкроковому режимі (half-step) кількість станів збільшується вдвічі, чергуючи увімкнення однієї та двох фаз. Послідовність виглядає так: `0x09`, `0x08`, `0x0C`, `0x04`, `0x06`, `0x02`, `0x03`, `0x01`. Це дозволяє збільшити точність позиціювання ротора вдвічі (кут кроку стає 45 градусів замість 90), що також чітко підтверджується зміненою формою сигналів на осцилограмі вище.

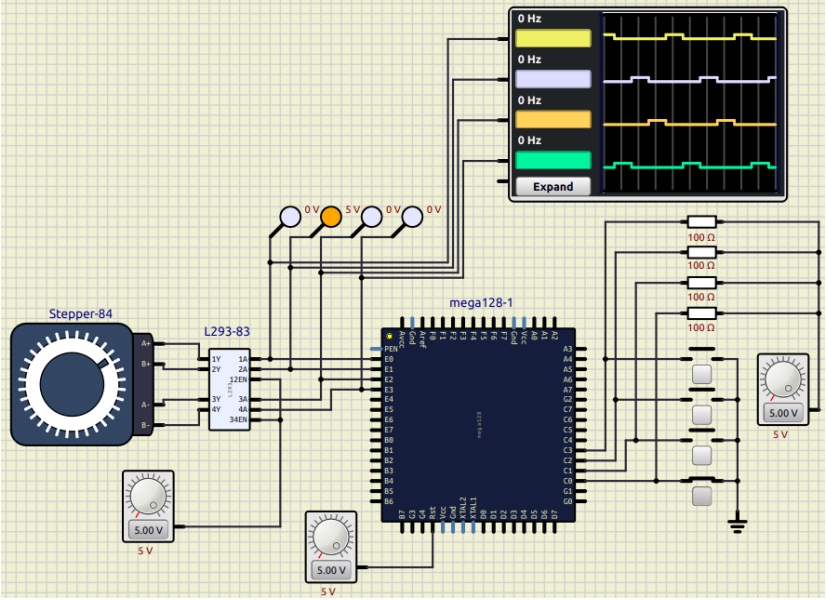
Переходимо до другої частини лабораторної роботи – схеми з біполярним двигуном. У налаштуваннях компонента `Stepper` тип двигуна був змінений на біполярний. Оскільки біполярний двигун не має відводів від середини обмоток і вимагає зміни полярності напруги для обертання, попередній драйвер був замінений на мікросхему `L293-83` (аналог L293D).

Цей драйвер містить Н-мостові схеми. Керуючі сигнали від мікроконтролера (порти PE0–PE3) підключаються до входів драйвера 1A, 2A, 3A та 4A. Силові виходи 1Y, 2Y, 3Y та 4Y підключаються безпосередньо до чотирьох контактів біполярного двигуна. Важливою відмінністю є необхідність подачі логічної одиниці (5 В) на контакти дозволу роботи мостів `12EN` та `34EN`, щоб драйвер міг пропускати струм на двигун.



Після збирання апаратної частини був скомпільований код для крокового режиму біполярного двигуна (лістинг `bipolar_step.c`). Для біполярного двигуна в однофазному режимі з цілим кроком напруга подається лише на одну обмотку в певний момент часу. Відповідна послідовність шістнадцяткових значень для порту E у коді виглядає так: `0x08` (1000), `0x02` (0010), `0x04` (0100) та `0x01` (0001).

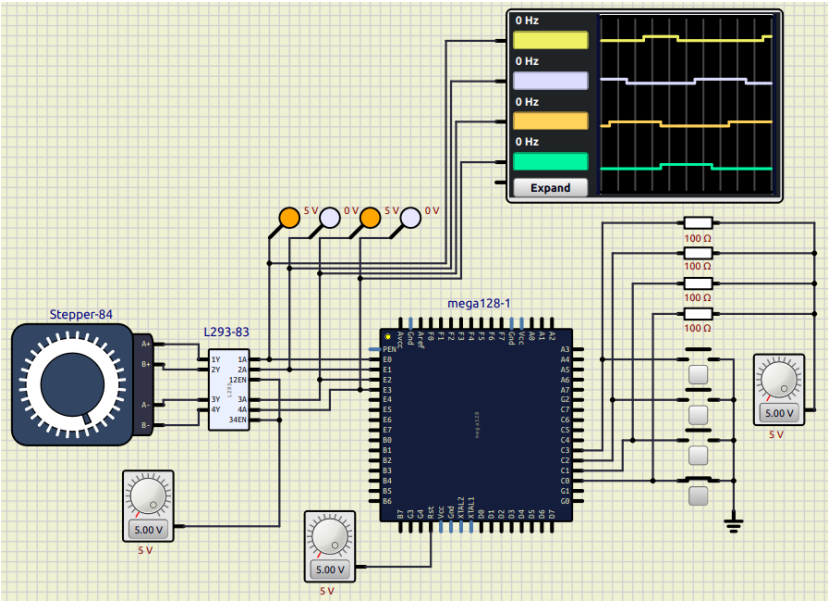
Прошивка була завантажена у мікроконтролер. Результати керуючих імпульсів (зсунуті по фазі короткі прямокутні сигнали), зафіксовані віртуальним осцилографом під час автоматичного обертання, наведено нижче:



Наступним кроком стало дослідження напівкрокового режиму роботи біполярного двигуна . Для цього був використаний та скомпільований останній файл з кодом – `bipolar_half_step.c`.

Цей режим поєднує в собі однофазне та двофазне живлення обмоток. Керуюча послідовність для забезпечення мікрокроку у 45 градусів вимагає восьми тактів: `0x01`, `0x09`, `0x08`, `0x0A`, `0x02`, `0x06`, `0x04`, `0x05`. Це найскладніший режим керування серед досліджуваних, який забезпечує найбільш плавний рух валу двигуна, але вимагає складнішого алгоритму контролера.

Отримані осцилограми для напівкрокового режиму біполярного двигуна демонструють характерне «східчасте» перекриття сигналів:



Висновок. Під час виконання лабораторної роботи було успішно досліджено принципи керування кроковими двигунами за допомогою мікроконтролера ATMega128 у середовищі моделювання SimulIDE. Було розроблено та протестовано дві апаратні схеми: перша для уніполярного крокового двигуна з використанням транзисторної збірки Дарлінгтона ULN2803, а друга для біполярного крокового двигуна на базі мостового драйвера L293. Для кожної схеми було створено програмне забезпечення мовою C, яке реалізує генерацію керуючих імпульсів. Практично перевірено роботу двигунів у повнокроковому та напівкроковому режимах. За допомогою віртуального осцилографа було зафіксовано та проаналізовано форми сигналів, що підтвердило правильність розроблених алгоритмів. Зміна режимів роботи наочно продемонструвала різницю у плавності ходу та куті повороту ротора за один такт керування, а також довела необхідність використання спеціалізованих силових мікросхем-драйверів для узгодження слабкострумових виходів мікроконтролера з потужними обмотками двигунів.

Контрольні запитання.

1. **Дайте визначення уніполярним кроковим двигунам.**
   Уніполярний кроковий двигун – це електромеханічний пристрій, кожна фазна обмотка якого має відведення від середини. Така конструкція дозволяє створювати магнітні поля різної полярності шляхом подачі струму лише в одному напрямку на відповідну половину обмотки, що значно спрощує схему керування, оскільки зникає необхідність апаратного перемикання полярності напруги.
2. **Дайте визначення біполярним кроковим двигунам.**
   Біполярний кроковий двигун – це тип двигуна, який має по одній суцільній обмотці на кожну фазу без центральних відводів. Для зміни напрямку магнітного поля в такому двигуні драйвер повинен фізично змінювати напрямок протікання струму через всю котушку, що вимагає складнішої схемотехніки керування, але дозволяє ефективніше використовувати об’єм міді та отримувати більший крутний момент.
3. **Як уніполярний кроковий двигун підключається до мікроконтролера?**
   Через великий струм споживання двигун не можна підключати до мікроконтролера безпосередньо. Він підключається через силовий драйвер, який містить масив транзисторних ключів. Інформаційні виводи мікроконтролера з’єднуються з логічними входами драйвера, силові виходи драйвера підключаються до кінців фазних обмоток двигуна, а загальний центральний

провід двигуна під'єднується безпосередньо до окремого джерела живлення двигуна.

4. **Опишіть роботу драйвера уніполярного крокового двигуна.**

   Драйвер уніполярного крокового двигуна працює як набір електронних перемикачів. Коли мікроконтролер подає логічну одиницю на вхід драйвера, відповідний внутрішній транзистор відкривається і замикає підключену до його виходу половину фазної обмотки двигуна на землю. Це дозволяє потужному струму від джерела живлення пройти через котушку, створюючи магнітне поле, при цьому слабкострумові кола мікроконтролера залишаються повністю захищеними.

5. **Назвіть та опишіть способи керування кроковими двигунами.**

   Існує кілька основних способів керування. Однофазний кроковий режим передбачає подачу напруги лише на одну фазу в певний момент часу, що забезпечує базовий крок, але найменший крутний момент. Двофазний кроковий режим передбачає одночасне збудження двох фаз, що дає такий самий кут кроку, але значно більшу потужність і фіксацію. Напівкроковий режим чергує увімкнення однієї та двох фаз, що дозволяє зменшити кут кроку вдвічі та зробити рух плавнішим. Мікрокроковий режим використовує широтно-імпульсну модуляцію для плавного регулювання струму в обмотках, розбиваючи один фізичний крок на десятки мікрокроків для досягнення максимальної плавності та точності.

6. **Наведіть та пояснить схему алгоритму роботи та робочу програму моделі уніполярного крокового двигуна для крокового режиму роботи.**

   Алгоритм працює у нескінченному циклі, постійно опитуючи стан кнопок керування. При активації автоматичного або ручного режиму мікроконтролер послідовно видає в порт керування шістнадцяткові значення 0x0C, 0x06, 0x03 та 0x09. Кожне з цих значень одночасно встановлює високий рівень на двох ніжках порту, відкриваючи відповідні ключі драйвера. Після кожної зміни стану викликається функція програмної затримки, яка визначає швидкість обертання ротора від одного повнокрокового положення до іншого.

7. **Наведіть та пояснить схему алгоритму роботи та робочу програму моделі уніполярного крокового двигуна для напівкрокового режиму роботи.**

   Програмна логіка напівкрокового режиму ідентична повнокроковому в плані опитування кнопок та використання затримок, але масив керуючих сигналів розширюється вдвічі. Тепер програма генерує вісім послідовних станів:

0x09, 0x08, 0x0C, 0x04, 0x06, 0x02, 0x03 та 0x01. Ця послідовність почергово вмикає то дві сусідні фази, то лише одну, завдяки чому ротор фіксується не лише навпроти полюсів статора, а й у проміжних положеннях між ними.

8. **Поясніть результати моделювання уніполярного та біполярного крокових двигунів.**

   Моделювання успішно підтвердило теоретичні принципи роботи обох типів двигунів. На осцилограмах чітко видно, що мікроконтролер генерує правильні послідовності прямокутних імпульсів, які зсунуті по фазі один відносно одного. Уніполярний двигун здійснював обертання при простій подачі напруги на різні канали, тоді як біполярний двигун коректно відпрацьовував сигнали від Н-мостового драйвера, демонструючи очікувані зміни полярності. Напівкрокові режими в обох випадках візуалізували збільшення частоти перемикань та перекриття імпульсів.

9. **Як біполярний кроковий двигун підключається до мікроконтролера?**

   Біполярний двигун підключається через мікросхему мостового драйвера, що містить транзисторні Н-мости. Логічні виходи мікроконтролера приєднуються до входів напрямку струму драйвера. Кожна з двох обмоток двигуна підключається до пари силових виходів відповідного Н-моста. Для функціонування схеми на контакти дозволу роботи драйвера також має бути постійно поданий високий логічний рівень напруги.

10. **Опишіть роботу драйвера біполярного крокового двигуна.**

    Драйвер біполярного двигуна, маючи всередині Н-мостову структуру, здатний не просто подавати струм в одному напрямку, а повністю змінювати його полярність на кінцях підключеної котушки. Залежно від комбінації логічних рівнів нулів та одиниць на входах керування, внутрішня логіка мікросхеми відкриває діагональні пари транзисторів, пропускаючи струм через обмотку зліва направо або справа наліво, що і призводить до зміни магнітного полюса статора.

11. **Опишіть алгоритм та робочу програму моделі біполярного крокового двигуна.**

    Алгоритм керування біполярним двигуном зберігає ту саму архітектуру нескінченного циклу з опитуванням контактів, але використовує специфічні послідовності бітів для правильної ініціалізації Н-мостів. Для найпростішого хвильового крокового режиму програма видає коди 0x08, 0x02, 0x04 та 0x01. Ці команди вказують драйверу, яку конкретно обмотку потрібно увімкнути і в якому саме напрямку пропустити через неї струм, щоб притягну-

ти ротор до наступної позиції.

12. **Дайте порівняльну характеристику крокових двигунів.**

Уніполярні двигуни відрізняються максимальною простотою керуючої електроніки, оскільки вимагають лише простих транзисторних ключів без необхідності зміни полярності струму, однак у будь-який момент часу працює лише половина їхніх обмоток, що робить їх менш ефективними. Біполярні двигуни використовують увесь об'єм міді своїх котушок, завдяки чому забезпечують значно вищий крутний момент при аналогічних габаритах, проте їхнє підключення потребує складніших та дорожчих мостових драйверів.

Лістинг.

unipolar_step.c

```c
// Define CPU frequency as 16 MHz for the delay library
#define F_CPU 16000000UL

#include <avr/io.h>
#include <util/delay.h>

#define STEP_UP_AUTO (PINC & (1 << PINC0))
#define STEP_DOWN_AUTO (PINC & (1 << PINC1))
#define STEP_UP (PINC & (1 << PINC2))
#define STEP_DOWN (PINC & (1 << PINC3))

int check_up;
int check_down;
int angle;

void stepper() {
    if (angle == 90)
    PORTE = 0x0C;
    if (angle == 180)
    PORTE = 0x06;
    if (angle == 270)
    PORTE = 0x03;
    if (angle == 360)
    PORTE = 0x09;
    if (angle == 0)
    PORTE = 0x09;
    if (angle == -90)
    PORTE = 0x03;
    if (angle == -180)
    PORTE = 0x06;
    if (angle == -270)
    PORTE = 0x0C;
    if (angle == -360)
    PORTE = 0x09;
}

void init_ports() {
    DDRC = 0x00;
    DDRE = 0xFF;
}

int main(void) {
    // Initialize variables here to avoid error
    // ELPM with no RAMPZ
    check_up = 1;
    check_down = 1;
    angle = 0;

    init_ports();
    while (1) {
        if (!STEP_UP_AUTO) {
            PORTE = 0x0C;
            _delay_ms(1000);
            PORTE = 0x06;
            _delay_ms(1000);
            PORTE = 0x03;
            _delay_ms(1000);
            PORTE = 0x09;
```

```
            _delay_ms(1000);
        }
        if (!STEP_DOWN_AUTO) {
            PORTE = 0x03;
            _delay_ms(1000);
            PORTE = 0x06;
            _delay_ms(1000);
            PORTE = 0x0C;
            _delay_ms(1000);
            PORTE = 0x09;
            _delay_ms(1000);
        }
        if (!STEP_UP && check_up == 1) {
            if (angle == 360) {
                angle = 0;
            }
            angle += 90;
            check_up = 0;
        }
        if (STEP_UP) {
            check_up = 1;
        }
        if (!STEP_DOWN && check_down == 1) {
            if (angle == -360) {
                angle = 0;
            }
            angle -= 90;
            check_down = 0;
        }
        if (STEP_DOWN) {
            check_down = 1;
        }
        stepper();
    }
}
```

unipolar_half_step.c

```
// Define CPU frequency as 16 MHz for the delay library
#define F_CPU 16000000UL

#include <avr/io.h>
#include <util/delay.h>

#define STEP_UP_AUTO (PINC & (1 << PINC0))
#define STEP_DOWN_AUTO (PINC & (1 << PINC1))
#define STEP_UP (PINC & (1 << PINC2))
#define STEP_DOWN (PINC & (1 << PINC3))

int check_up;
int check_down;
int angle;

void stepper() {
    if (angle == 0)
    PORTE = 0x09;
    if (angle == 45)
    PORTE = 0x08;
    if (angle == 90)
    PORTE = 0x0;
    if (angle == 135)
    PORTE = 0x04;
```

```
    if (angle == 180)
    PORTE = 0x06;
    if (angle == 225)
    PORTE = 0x02;
    if (angle == 270)
    PORTE = 0x03;
    if (angle == 315)
    PORTE = 0x01;
    if (angle == 360)
    PORTE = 0x09;
    if (angle == -45)
    PORTE = 0x01;
    if (angle == -90)
    PORTE = 0x03;
    if (angle == -135)
    PORTE = 0x02;
    if (angle == -180)
    PORTE = 0x06;
    if (angle == -225)
    PORTE = 0x04;
    if (angle == -270)
    PORTE = 0x0C;
    if (angle == -315)
    PORTE = 0x08;
    if (angle == -360)
    PORTE = 0x09;
}

void init_ports() {
    DDRC = 0x00;
    DDRE = 0xFF;
}

int main(void) {
    // Initialize variables here to avoid error
    // ELPM with no RAMPZ
    check_up = 1;
    check_down = 1;
    angle = 0;

    init_ports();
    while (1) {
        if (!STEP_UP_AUTO) {
            PORTE = 0x08;
            _delay_ms(1000);
            PORTE = 0x0C;
            _delay_ms(1000);
            PORTE = 0x04;
            _delay_ms(1000);
            PORTE = 0x06;
            _delay_ms(1000);
            PORTE = 0x02;
            _delay_ms(1000);
            PORTE = 0x03;
            _delay_ms(1000);
            PORTE = 0x01;
            _delay_ms(1000);
            PORTE = 0x09;
            //_delay_ms(1000);
        }
        if (!STEP_DOWN_AUTO) {
            PORTE = 0x01;
            _delay_ms(1000);
```

```
            PORTE = 0x03;
            _delay_ms(1000);
            PORTE = 0x02;
            _delay_ms(1000);
            PORTE = 0x06;
            _delay_ms(1000);
            PORTE = 0x04;
            _delay_ms(1000);
            PORTE = 0x0C;
            _delay_ms(1000);
            PORTE = 0x08;
            _delay_ms(1000);
            PORTE = 0x09;
            //_delay_ms(1000);
        }
        if (!STEP_UP && check_up == 1) {
            if (angle == 360) {
                angle = 0;
            }
            angle += 45;
            check_up = 0;
        }
        if (STEP_UP) {
            check_up = 1;
        }
        if (!STEP_DOWN && check_down == 1) {
            if (angle == -360) {
                angle = 0;
            }
            angle -= 45;
            check_down = 0;
        }
        if (STEP_DOWN) {
            check_down = 1;
        }
        stepper();
    }
}
```

bipolar_step.c

```
// Define CPU frequency as 16 MHz for the delay library
#define F_CPU 16000000UL

#include <avr/io.h>
#include <util/delay.h>

#define STEP_UP_AUTO (PINC & (1 << PINC0))
#define STEP_DOWN_AUTO (PINC & (1 << PINC1))
#define STEP_UP (PINC & (1 << PINC2))
#define STEP_DOWN (PINC & (1 << PINC3))

int check_up;
int check_down;
int angle;

void stepper() {
    if (angle == 90)
    PORTE = 0x08;
    if (angle == 180)
    PORTE = 0x02;
    if (angle == 270)
```

```
    PORTE = 0x04;
    if (angle == 360)
    PORTE = 0x01;
    if (angle == 0)
    PORTE = 0x01;
    if (angle == -90)
    PORTE = 0x04;
    if (angle == -180)
    PORTE = 0x02;
    if (angle == -270)
    PORTE = 0x08;
    if (angle == -360)
    PORTE = 0x01;
}

void init_ports() {
    DDRC = 0x00;
    DDRE = 0xFF;
}

int main(void) {
    // Initialize variables here to avoid error
    // ELPM with no RAMPZ
    check_up = 1;
    check_down = 1;
    angle = 0;

    init_ports();
    while (1) {
        if (!STEP_UP_AUTO) {
            PORTE = 0x08;
            _delay_ms(1000);
            PORTE = 0x02;
            _delay_ms(1000);
            PORTE = 0x04;
            _delay_ms(1000);
            PORTE = 0x01;
            _delay_ms(1000);
        }
        if (!STEP_DOWN_AUTO) {
            PORTE = 0x04;
            _delay_ms(1000);
            PORTE = 0x02;
            _delay_ms(1000);
            PORTE = 0x08;
            _delay_ms(1000);
            PORTE = 0x01;
            _delay_ms(1000);
        }
        if (!STEP_UP && check_up == 1) {
            if (angle == 360) {
                angle = 0;
            }
            angle += 90;
            check_up = 0;
        }
        if (STEP_UP) {
            check_up = 1;
        }
        if (!STEP_DOWN && check_down == 1) {
            if (angle == -360) {
                angle = 0;
            }
```

```
            angle -= 90;
            check_down = 0;
        }
        if (STEP_DOWN) {
            check_down = 1;
        }
        stepper();
    }
}
```

bipolar_half_step.c

```
// Define CPU frequency as 16 MHz for the delay library
#define F_CPU 16000000UL

#include <avr/io.h>
#include <util/delay.h>

#define STEP_UP_AUTO (PINC & (1 << PINC0))
#define STEP_DOWN_AUTO (PINC & (1 << PINC1))
#define STEP_UP (PINC & (1 << PINC2))
#define STEP_DOWN (PINC & (1 << PINC3))

int check_up;
int check_down;
int angle;

void stepper() {
    if (angle == 0)
    PORTE = 0x01;
    if (angle == 45)
    PORTE = 0x09;
    if (angle == 90)
    PORTE = 0x08;
    if (angle == 135)
    PORTE = 0x0A;
    if (angle == 180)
    PORTE = 0x02;
    if (angle == 225)
    PORTE = 0x06;
    if (angle == 270)
    PORTE = 0x04;
    if (angle == 315)
    PORTE = 0x05;
    if (angle == 360)
    PORTE = 0x01;
    if (angle == -45)
    PORTE = 0x05;
    if (angle == -90)
    PORTE = 0x04;
    if (angle == -135)
    PORTE = 0x06;
    if (angle == -180)
    PORTE = 0x02;
    if (angle == -225)
    PORTE = 0x0A;
    if (angle == -270)
    PORTE = 0x08;
    if (angle == -315)
    PORTE = 0x09;
    if (angle == -360)
    PORTE = 0x01;
```

```
}

void init_ports() {
    DDRC = 0x00;
    DDRE = 0xFF;
}

int main(void) {
    // Initialize variables here to avoid error
    // ELPM with no RAMPZ
    check_up = 1;
    check_down = 1;
    angle = 0;

    init_ports();
    while (1) {
        if (!STEP_UP_AUTO) {
            PORTE = 0x09;
            _delay_ms(1000);
            PORTE = 0x08;
            _delay_ms(1000);
            PORTE = 0x0A;
            _delay_ms(1000);
            PORTE = 0x02;
            _delay_ms(1000);
            PORTE = 0x06;
            _delay_ms(1000);
            PORTE = 0x04;
            _delay_ms(1000);
            PORTE = 0x05;
            _delay_ms(1000);
            PORTE = 0x01;
            //_delay_ms(1000);
        }
        if (!STEP_DOWN_AUTO) {
            PORTE = 0x05;
            _delay_ms(1000);
            PORTE = 0x04;
            _delay_ms(1000);
            PORTE = 0x06;
            _delay_ms(1000);
            PORTE = 0x02;
            _delay_ms(1000);
            PORTE = 0x0A;
            _delay_ms(1000);
            PORTE = 0x08;
            _delay_ms(1000);
            PORTE = 0x09;
            _delay_ms(1000);
            PORTE = 0x01;
            //_delay_ms(1000);
        }
        if (!STEP_UP && check_up == 1) {
            if (angle == 360) {
                angle = 0;
            }
            angle += 45;
            check_up = 0;
        }
        if (STEP_UP) {
            check_up = 1;
        }
        if (!STEP_DOWN && check_down == 1) {
```

```
            if (angle == -360) {
                angle = 0;
            }
            angle -= 45;
            check_down = 0;
        }
        if (STEP_DOWN) {
            check_down = 1;
        }
        stepper();
    }
}
```